Name: Dau Vu Tuan Khoi

ID: 105709548

C3 Report

1. Summary of Research and Code Explaination

In version v0.2 of the project, the primary objective is to implement data visualization techniques to add more value to the analysis. To fulfill the requirements of generating a candlestick chart and a boxplot chart, I had to research and apply several advanced programming logics.

Using the specialized mplfinance library: Instead of attempting to building a candlestick chart from scratch using Matplotlib, I researched and utilized the mplfinance library. The core command mpf.plot(…, type='candle', style='charles', volume=True) was referenced directly from online documentation. It effectively renders candlesticks with traditional colors and automatically integrates the trading volume band.

N-day grouping logic (Candlestick chart): The project requires allowing each candle to represent the data of n trading days (n \ge 1). Instead of using Pandas' standard time-based resample function (which often causes errors due to market closures on weekends), I utilized integer division: *np.arange(len(df)) //n_days*. This code counts the actual row indices to group the data, ensuring that each candle strictly aggregates exactly the n days of actual trading activity.

Array reshaping for sliding windows (Boxplot): To display the data distribution for a sliding window of n consecutive trading days, the data format had to be transformed. I used Numpy's reshape function via the command *truncated_prices.reshape((num_windows, window_size))*. This required specific calculations to truncate the excess data points at the end, ensuring the 1D array folded perfectly into a 2D matrix. In the matrix, each column represents an independent time window that can be passed directly into the *plt_boxplot* function.

2. Online Resources Referenced

Guide on plotting financial candlestick charts: https://coderzcolumn.com/tutorials/data-science/candlestick-chart-in-python-mplfinance-plotly-bokeh.

Official Numpy documentation (utilized for numpy.arange and numpy.reshape).

Official Matplotlib documentation (utilized for instructions on matplotlib.pyplot.boxplot).

3. Main Challenges Faced

While accomplishing this task, the most significant challenge stemmed from the non-linear nature of financial time-series data.

Specifically, the stock market has time gaps since it does not operate on weekends and public holidays. Initially, when I tried to set up sliding windows or aggregate n-days based on standard datetime functions, the charts were constantly disorted or generated “black candles.” Switching the approach to process data based on array row indices (as explained in Section 1) was the troubleshooting step that required the most research effort.

Furthermore, when plotting the boxplot, accurately aligning the X-asis labels to correspond with the start date of each sliding window proved to be another hurdle. It required very careful slicing and extraction of the index series to perfectly match the 1:1 ratio with the number of boxes displayed on the chart.